

Deep Learning Avancé & Applications

Rappel CNN & Architectures Avancées

Master Bac+5 – Data Analytics & Big Data

Mouaad AGOURRAM

2025–2026

Golden Collar

Rappel : Réseaux de Neurones Convolutifs (CNN)

Architectures CNN Avancées : VGG & ResNet

Transfer Learning

Fine-Tuning : Techniques Avancées

Vision par Ordinateur : Applications et Pipeline

Synthèse & TP

Rappel : Réseaux de Neurones Convolutifs (CNN)

Objectifs pédagogiques de cette séance

Objectif principal

Maîtriser les **architectures CNN avancées** et exploiter la **puissance du Transfer Learning** pour résoudre des problèmes réels de vision par ordinateur.

À l'issue de cette séance, vous serez capable de :

- Rappeler et expliquer les composants fondamentaux d'un **CNN**
- Comprendre et implémenter les architectures **VGG** et **ResNet**
- Appliquer le **Transfer Learning** avec des modèles pré-entraînés
- Réaliser du **Fine-tuning** adapté à vos données
- Concevoir un pipeline complet de **vision par ordinateur**

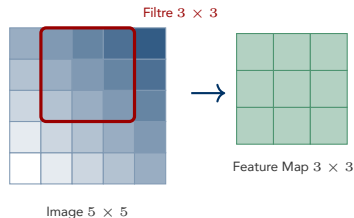
Intuition : pourquoi les CNN dominent la vision ?

Principe clé : hiérarchie de représentations

- **Couches basses** : détecteurs de bords, couleurs, textures simples
- **Couches intermédiaires** : formes, coins, motifs locaux
- **Couches hautes** : objets, concepts sémantiques

Avantages structurels

- **Partage de paramètres** : même filtre appliqué sur toute l'image
- **Invariance locale** : robustesse aux translations
- **Connexions locales** : exploitation du voisinage spatial
- **Efficacité** : bien moins de paramètres qu'un



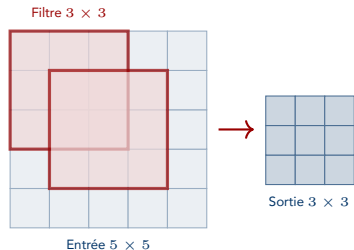
Formule convolution 2D

$$(I * K)(i, j) = \sum_{m, n} I(i+m, j+n) \cdot K(m, n)$$

Concept

- Filtre $K \times K$ glissant sur l'entrée
- **Stride** s : pas du filtre
- **Padding** p : gestion des bords
- Taille sortie : $\left\lfloor \frac{n + 2p - k}{s} + 1 \right\rfloor$
- Partage de poids $\Rightarrow$ peu de paramètres
- Détection de motifs locaux (bords, textures...)

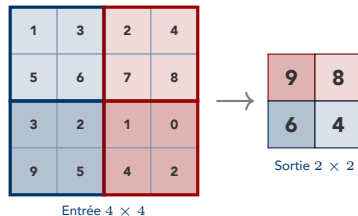
Illustration : filtre 3×3 sur entrée 5×5



Concept

- Réduit la dimension spatiale
- **Max Pooling** : valeur maximale
⇒ invariance locale aux translations
- **Average Pooling** : moyenne
⇒ lissage des activations
- **Global Average Pooling (GAP)** :
réduit chaque feature map à un scalaire
⇒ évite le sur-apprentissage du FC

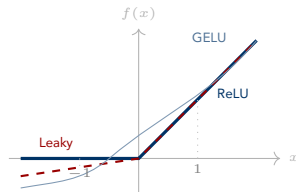
Illustration : Max Pooling 2×2 , stride 2



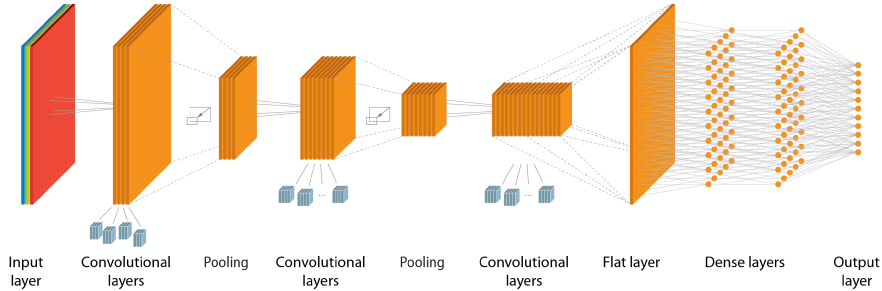
Concept

- Introduit la **non-linéarité** nécessaire
- **ReLU** : $f(x) = \max(0, x)$
simple, pas de gradient vanishing
- **Leaky ReLU** : $f(x) = \max(\alpha x, x)$
évite les neurones morts
- **GELU / SiLU** : fonctions lisses
utilisées dans Transformers et EfficientNet

Illustration : courbes d'activation



Composants CNN (4/4) : Couche Fully Connected



Concept

- Placée après Flatten ou GAP
- Chaque neurone connecté à toutes les entrées
- Combine les features spatiales extraites
- **Softmax** en sortie $\Rightarrow$ distribution de probabilité
- Attention au sur-apprentissage : **Dropout** recommandé

Hyperparamètres clés d'une couche de convolution

Hyperparamètre	Rôle	Valeurs typiques
Nombre de filtres	Richesse de représentation	32, 64, 128, 256, 512
Taille du filtre	Champ réceptif local	3×3 (standard), 5×5 , 1×1
Stride	Réduction spatiale implicite	1 (standard), 2 (downsampling)
Padding	Conservation des dimensions	<code>same</code> (conserve) / <code>valid</code> (réduit)
Activation	Non-linéarité	ReLU, GELU, SiLU
Dilatation	Champ réceptif élargi sans stride	1 (std), 2, 4 (trous conv)

Règle empirique fondamentale

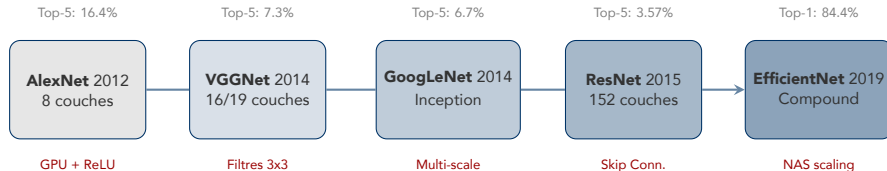
Deux convolutions 3×3 = champ réceptif 5×5 avec **moins de paramètres** et **plus de non-linéarité**. C'est le principe fondateur de VGG.

Convolution 1×1 : rôle clé

Modifier le nombre de canaux **sans toucher aux dimensions spatiales**. Utilisée massivement dans les blocs bottleneck de ResNet.

Architectures CNN Avancées : VGG & ResNet

L'évolution des architectures CNN (2012-2021)



La grande leçon : innovation architecturale > simple profondeur

Chaque architecture a apporté une **innovation clé** permettant d'aller plus profond.

Franchissement du seuil humain (5.1%) par ResNet en 2015

Avec ResNet, les machines surpassent officiellement l'humain sur ImageNet pour la classification.

Architecture VGGNet : principes de conception

Design Philosophy (Simonyan & Zisserman, 2014)

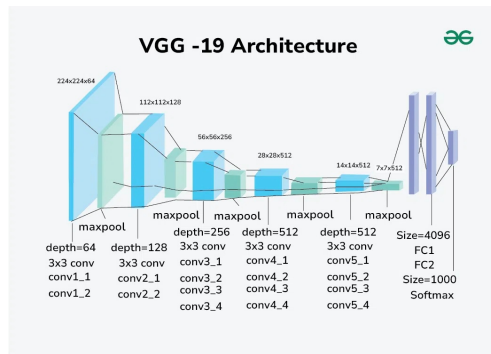
- **Uniformité** : uniquement des filtres 3×3 , stride 1, padding same
- **Doublement** des canaux : $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$
- **Max Pooling** 2×2 pour réduire la résolution ($\div 2$)
- **3 couches FC** : $4096 \rightarrow 4096 \rightarrow 1000$

Deux variantes populaires

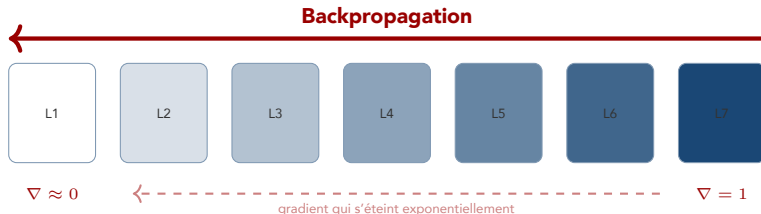
- **VGG-16** : 13 conv + 3 FC = **138M paramètres**
- **VGG-19** : 16 conv + 3 FC = **144M paramètres**

Problème majeur

92% des paramètres concentrés dans les 3



Le problème du gradient vanishing



Vanishing Gradient

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \prod_{k=1}^n \frac{\partial h_k}{\partial h_{k-1}}$$

Si chaque facteur < 1 : gradient $\rightarrow 0$ exponentiellement. Les premières couches **n'apprennent plus**.

Conséquences observées

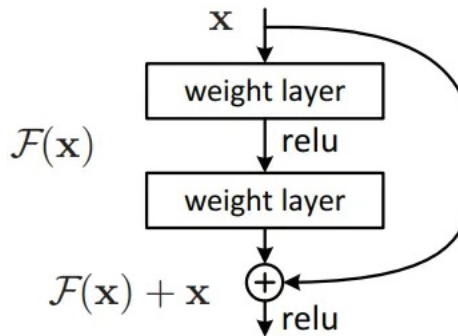
- Convergence lente ou nulle
- Performance inférieure à des réseaux moins profonds
- Instabilité numérique
- Avant 2015 : impossible de dépasser ~ 20 couches

Le Residual Block (He et al., 2015)

Au lieu d'apprendre $H(x)$ directement, on apprend le **résidu** $F(x) = H(x) - x$:

$$y = F(x, \{W_i\}) + x$$

Le bloc apprend seulement la **différence** par rapport à l'identité.



Pourquoi ça marche ?

- **Gradient direct** : le signal remonte par le shortcut sans atténuation
- **Solution triviale** : $F(x) = 0$ donne l'identité (initialisation stable)
- **Profondeur** : permet 50, 101, 152, voire **1000+ couches**
- **Entraînabilité** : les couches précoces reçoivent un gradient utile

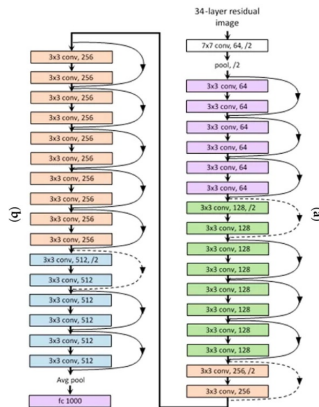


Figure 1: ResNet-34 Layered architecture

Basic Block (ResNet-18, ResNet-34)

Deux convolutions 3×3 empilées :

$$\text{Conv}_{3 \times 3} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}_{3 \times 3} \rightarrow \text{BN} \oplus x$$

- Paramètres par bloc : $18C^2$
- Adapté aux petits modèles (mobile, edge)

Bottleneck Block (ResNet-50+)

Compression $\rightarrow$ traitement $\rightarrow$ expansion :

$$\text{Conv}_{1 \times 1} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}_{3 \times 3} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}_{1 \times 1} \rightarrow \text{BN} \oplus x$$

- 1×1 : réduit de C à $C/4$ canaux
- 3×3 : traitement sur espace réduit
- **3x moins de calculs** à même champ réceptif

Modèle	Couches	Param.	Top-1	Bloc	Usage
ResNet-18	18	11.7M	70.0%	Basic	Mobile / Edge
ResNet-34	34	21.8M	73.3%	Basic	Usage général
ResNet-50	50	25.6M	76.1%	Bottleneck	Standard prod.
ResNet-101	101	44.5M	77.4%	Bottleneck	Haute préc.
ResNet-152	152	60.2M	78.3%	Bottleneck	Compétitions

VGG vs ResNet : comparaison pratique

Critère	VGG-16	ResNet-50
Paramètres totaux	138M	25.6M
Top-1 ImageNet (val)	71.3%	76.1%
Profondeur	16 couches	50 couches
Mémoire GPU (inférence)	Très élevée	Modérée
Vitesse inférence	Lente	Rapide
Transférabilité des features	Très bonne	Excellente
Batch Normalization	Non (d'origine)	Oui
Facilité de fine-tuning	Bonne	Excellente
Usage recommandé	Benchmarks, Neural Style	Production & Transfer Learning

Conclusion pratique 2025

En production, utilisez **ResNet-50** ou **EfficientNet-B3**. VGG reste utile pour le Neural Style Transfer et pour comprendre les fondations – mais pas en production (trop lourd).

Transfer Learning

Qu'est-ce que le Transfer Learning ?

Définition

Le **Transfer Learning** consiste à **réutiliser les poids** d'un modèle entraîné sur une grande tâche source pour accélérer l'apprentissage sur une tâche cible avec peu de données.

Pourquoi c'est puissant ?

- **Données limitées** : moins de données nécessaires ($\times 100$)
- **Performance** : résultats plus rapides et précis
- **Efficacité** : temps d'entraînement réduit ($\times 10$)
- **Adaptabilité** : de la vision à l'NLP

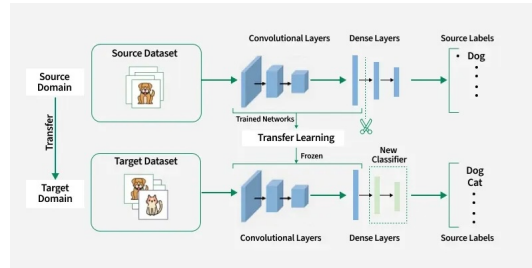


Figure 2: Transfer Learning illustration

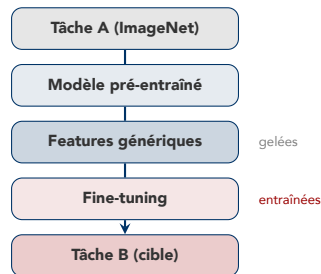
Comment fonctionne le Transfer Learning ?

Processus en 4 étapes

1. **Modèle pré-entraîné** : modèle entraîné sur grand dataset (ex. ImageNet 1.2M images)
2. **Modèle de base** : couches capturant des représentations hiérarchiques (bas → haut niveau)
3. **Couches transférables** : couches proches du sommet contenant des features génériques réutilisables
4. **Fine-tuning** : ajustement des paramètres sur les données cibles tout en conservant la connaissance pré-apprise

Principe clé

Les features bas niveau (bords, textures) appris sur la tâche A sont utiles pour la tâche B.



Aspect	Couches gelées	Couches entraînables
Définition	Poids fixés, non mis à jour	Poids mis à jour pendant l'entraînement
Objectif	Préserver les features générales	Adapter aux features spécifiques
Coût calcul	Faible (moins de paramètres)	Élevé (plus de paramètres)
Cas d'usage	Dataset petit / similaire	Dataset grand / différent
Exemple CNN	Couches conv. précoces (bords, textures)	Couches FC ou conv. profondes

Quelle stratégie selon les données ?

- **Petit + similaire** : geler presque tout, fine-tuner 1–2 dernières couches
- **Grand + similaire** : dégeler plus de couches
- **Petit + différent** : fine-tuner les couches proches de l'entrée
- **Grand + différent** : fine-tuner tout le modèle

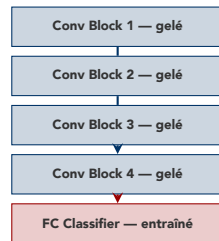
Stratégie 1 : Feature Extraction

Principe

- Backbone **complètement gelé** (poids non modifiés)
- Seule la tête de classification est entraînée
- Learning Rate standard : 10^{-3}
- Rapide, stable, faible risque d'overfitting
- Idéal si données similaires à ImageNet

Quand l'utiliser ?

Dataset petit (< 1000 images) **ET** similaire au domaine source (photos naturelles, objets quotidiens).



LR = 10^{-3}

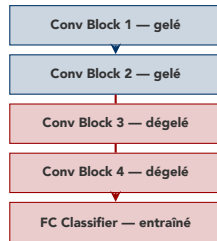
Stratégie 2 : Fine-Tuning

Principe

- Backbone **partiellement dégelé** (dernières couches)
- Backbone entraînable avec LR très petit : 10^{-5}
- Procéder **en 2 étapes** (Feature Extraction d'abord !)
- Plus long, risque de *catastrophic forgetting*
- Idéal si le domaine diffère d'ImageNet

Procédure obligatoire

Ne **JAMAIS** faire du fine-tuning directement !
Toujours commencer par la Feature Extraction (5–10 epochs), puis dégeler progressivement avec LR minimal.



LR = 10^{-5}

Stratégie recommandée en 2 étapes

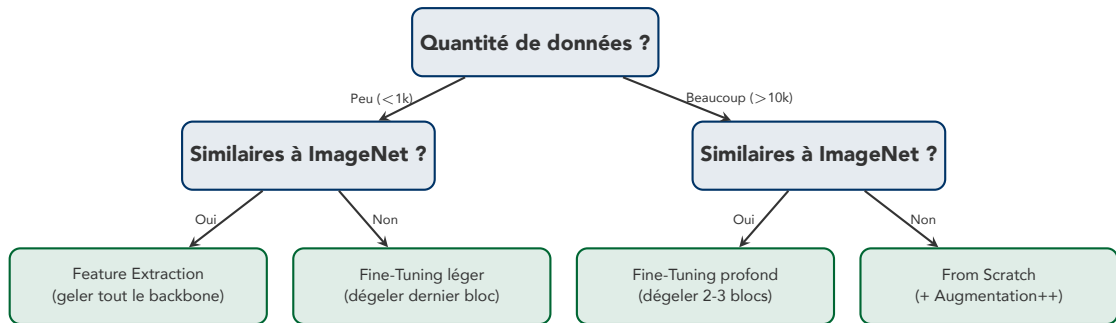
Étape 1 — Feature Extraction

- Geler tout le backbone
- Entraîner uniquement la tête de classification
- Durée : 5–10 epochs $LR = 10^{-3}$
- Objectif : converger rapidement vers une bonne représentation

Étape 2 — Fine-Tuning progressif

- Dégeler progressivement les dernières couches du backbone
- Continuer l'entraînement avec un LR très faible
- Durée : 10–20 epochs supplémentaires $LR = 10^{-5}$
- Objectif : affiner les features haut niveau pour la tâche cible

Arbre de décision : quelle stratégie choisir ?



Règle d'or

En pratique industrielle, 95% des cas se règlent avec **Feature Extraction + Fine-Tuning léger** sur ResNet-50 ou EfficientNet. Entraîner from scratch est réservé aux organisations ayant accès à d'immenses datasets (Google, Meta...).

Fine-Tuning : Techniques Avancées

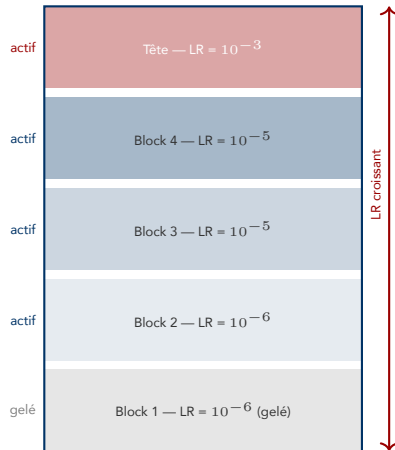
Principe (Discriminative Fine-Tuning)

Les différentes parties du réseau sont entraînées avec des **taux d'apprentissage différents** :

- **Couches précoces** : features universels — très petit LR ou gelées
- **Couches intermédiaires** : features spécifiques — petit LR
- **Couches finales + tête** : features à adapter — LR plus grand

Règle empirique

Diviser le LR par un facteur 10 à chaque groupe de couches en remontant depuis la sortie vers l'entrée.



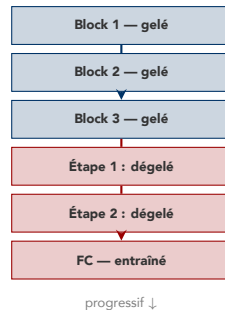
Technique 1 : Dégel progressif (Layer-wise Unfreezing)

Principe

- Dégeler les couches **de haut en bas**, progressivement
- Entraîner 2–3 epochs après chaque dégel
- Le réseau s'adapte progressivement à la tâche cible
- Limite fortement le *catastrophic forgetting*

Pourquoi de haut en bas ?

Les couches hautes capturent les features **spécifiques** à la tâche source. Ce sont elles qu'il faut adapter en premier. Les couches basses (bords, textures) restent utiles telles quelles.



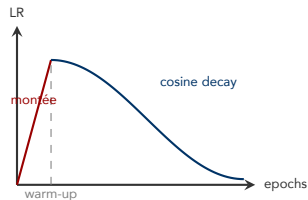
Technique 2 : Warm-up + Cosine Decay

Principe du scheduling

- Démarrer avec un LR très petit (**warm-up** : 5 epochs)
- Monter progressivement jusqu'au LR cible (10^{-4})
- Diminuer selon un **cosinus** jusqu'à ≈ 0
- Stabilise le début du fine-tuning, évite les oscillations

Formule Cosine Annealing

$$\text{LR}(t) = \text{LR}_{\min} + \frac{1}{2}(\text{LR}_{\max} - \text{LR}_{\min}) \left(1 + \cos \frac{\pi t}{T} \right)$$



Technique 3 : BatchNorm & Catastrophic Forgetting

BatchNorm : gestion spécifique

- BN en mode **inférence** pour les couches gelées
- Évite la corruption des statistiques pré-entraînées
- Si batch < 16 : utiliser **GroupNorm** à la place
- Keras : `layer.trainable = False` sur les BN gelés

Catastrophic Forgetting

Phénomène où l'apprentissage d'une nouvelle tâche **détruit** les connaissances précédentes.

Cause : LR trop grand sur les couches pré-entraînées.

Solution : $LR \leq 10^{-5}$ + dégel progressif + warm-up.

Règles BatchNorm

Couche	Mode BN
Gelée	Inférence (stats fixées)
Dégelée	Entraînement
Batch < 16	GroupNorm

Anti-forgetting

- Dégel couche par couche
- LR différentiel par groupe
- EWC (*Elastic Weight Consolidation*)

Étape 1 : Feature Extraction

```
base = ResNet50(  
    weights='imagenet',  
    include_top=False,  
    input_shape=(224,224,3))  
  
# Geler tout le backbone  
base.trainable = False  
  
x = base.output  
x = GlobalAvgPool2D()(x)  
x = Dense(256, activation='relu')(x)  
x = Dropout(0.3)(x)  
out = Dense(N, activation='softmax')(x)  
  
model = Model(base.input, out)  
model.compile(Adam(1e-3), ...)
```

Étape 2 : Fine-Tuning

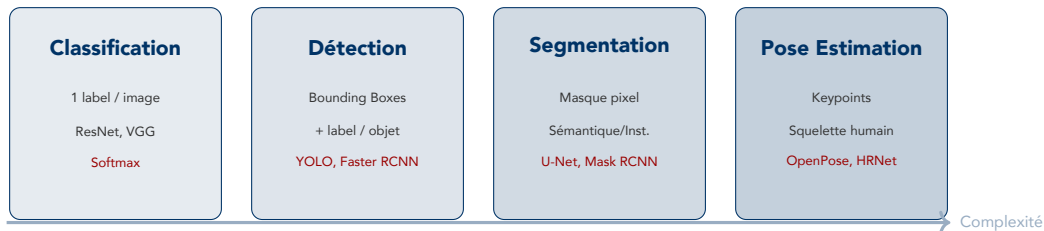
```
# Après convergence (10 epochs)  
base.trainable = True  
  
# Regeler les 100 premières couches  
for layer in base.layers[:100]:  
    layer.trainable = False  
  
# TOUJOURS recompiler après !  
model.compile(  
    Adam(1e-5),  
    loss='categorical_crossentropy',  
    metrics=['accuracy'])  
  
model.fit(train_ds, epochs=10, ...)
```

Point critique — TOUJOURS recompiler !

Après avoir changé `trainable`, Keras continue d'utiliser les anciens graphes de calcul si le modèle n'est pas recompilé.

Vision par Ordinateur : Applications et Pipeline

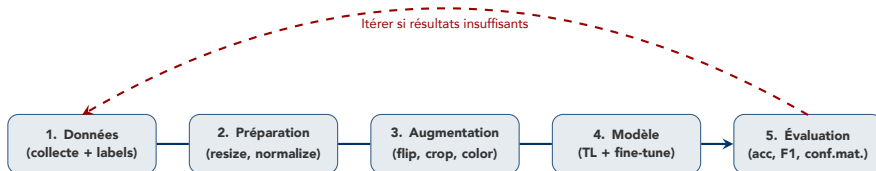
Les grandes tâches de Vision par Ordinateur



Paradigme commun : Backbone + Head

Toutes ces tâches utilisent un backbone CNN pré-entraîné (ResNet, EfficientNet, ViT) comme extracteur de features, suivi d'une **tête spécialisée par tâche**.

Pipeline complet de classification d'images



Bonnes pratiques Data

- Normaliser avec les stats ImageNet (μ , σ)
- Taille : 224×224 pour ResNet/VGG
- Split : 70/15/15 (train/val/test)
- Vérifier l'équilibre des classes
- Ne jamais toucher le test set avant la fin

Métriques d'évaluation

- **Accuracy** : simple, trompeuse si classes déséquilibrées
- **F1-score** : bon indicateur général
- **Matrice de confusion** : analyse fine
- **Top-5 accuracy** : utile pour beaucoup de classes
- **ROC-AUC** : évaluation multi-seuil

Transformations géométriques

- **Flip horizontal/vertical** : symétrie
- **Rotation** : $\pm 15^\circ$ à $\pm 30^\circ$
- **Zoom / RandomCrop** : 224 sur 256
- **Shear, Perspective** : déformations géométriques
- **Elastic transform** : utile en médical

Transformations colorimétriques

- Brightness, Contrast, Saturation, Hue
- Gaussian Blur, Gaussian Noise
- Grayscale aléatoire
- CLAHE : amélioration contraste local

Augmentations avancées

- **Mixup** : $\tilde{x} = \lambda x_i + (1-\lambda)x_j$
- **CutMix** : collage de régions entre images
- **CutOut** : masquage aléatoire de régions
- **RandAugment** : sélection automatique
- **AugMix** : mélange de plusieurs augmentations
- **AutoAugment** : appris par RL (Google)

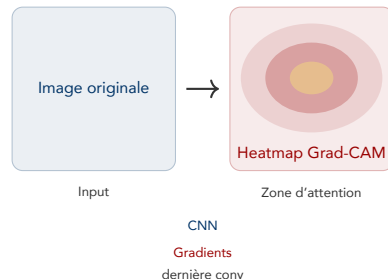
Règle d'or

L'augmentation ne doit pas modifier la **sémantique** de l'image. Exemple : ne pas retourner verticalement un chiffre "6" qui devient "9".

Gradient-weighted Class Activation Mapping

(Selvaraju et al., 2017)

Grad-CAM visualise **quelles régions** de l'image ont influencé la prédiction, en utilisant les gradients remontant vers la dernière couche de convolution.



Installation

```
pip install tf-keras-vis  
pip install grad-cam
```

Formule

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right)$$

α_k^c : importance du filtre k pour la classe c ,

A^k : carte d'activation du filtre k .

Utilisations pratiques

- Déboguer les erreurs de classification
- Valider que le modèle regarde la bonne région
- XAI : convaincre les stakeholders
- Détecter les biais du dataset

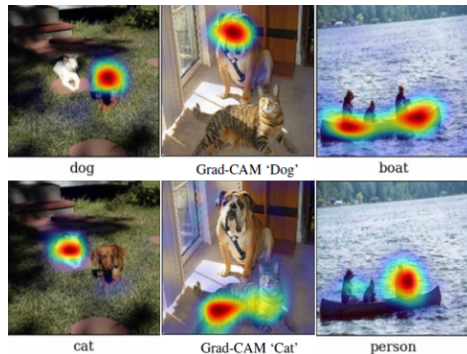


Figure 3: Visualisation Grad-CAM

Modèles pré-entraînés disponibles dans Keras

Modèle	Param.	Top-1	Taille	Usage recommandé
VGG16 / VGG19	138M / 144M	71.3%	Lourde	Benchmarks, Neural Style
ResNet50 / 101 / 152	25M / 44M / 60M	76.1% / 77.4% / 78.3%	Mod.	Standard production
InceptionV3	23.9M	78.0%	Mod.	Google Cloud Vision
MobileNetV2	3.5M	71.3%	Très légère	Mobile / Edge / IoT
EfficientNetB3	12M	81.6%	Légère	Best acc/param ratio
EfficientNetV2-M	54M	85.3%	Mod.	Haute précision
DenseNet121 / 201	8M / 20M	74.9% / 77.3%	Légère	Médical, peu de données
NASNetMobile	5.3M	74.4%	Légère	NAS, recherche

Accès en une ligne Keras

```
tf.keras.applications.ResNet50(weights='imagenet', include_top=False)
```

A retenir

Pour la plupart des projets : **EfficientNetB3** (meilleur ratio accuracy/paramètres) ou **ResNet50** (plus simple, mieux documenté).

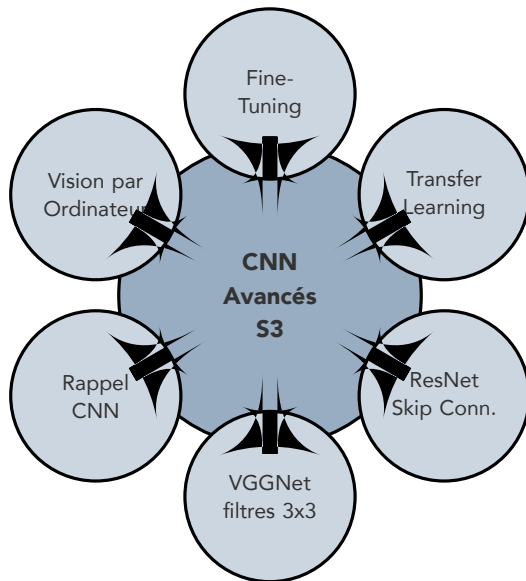
Synthèse & TP

Toujours faire

- Normaliser avec les stats ImageNet
- Commencer par la Feature Extraction
- Utiliser un LR très petit pour le fine-tuning ($\leq 10^{-5}$)
- Augmenter les données d'entraînement
- Vérifier avec Grad-CAM avant de déployer
- Monitorer train vs val loss à chaque epoch
- Utiliser EarlyStopping + ModelCheckpoint

Ne jamais faire

- Fine-tuner avec un grand LR (catastrophic forgetting)
- Oublier de geler le backbone en étape 1
- Appliquer des augmentations sémantiquement invalides
- Évaluer uniquement avec l'accuracy sur classes déséquilibrées
- Entraîner from scratch avec peu de données
- Oublier de recompiler après `trainable = True`
- Toucher le test set avant la fin de l'expérience



Objectif du TP

Implémenter un pipeline complet de classification d'images en utilisant le Transfer Learning avec **ResNet-50** sur le dataset **Oxford Flowers 102** (102 classes, ~8000 images).

Durée estimée et ressources

3 à 4 heures — GPU recommandé : Google Colab T4 (gratuit) ou Kaggle Kernel (30h GPU/semaine)

Questions ?

Séance suivante : *Architectures Séquentielles Avancées (LSTM, GRU, Attention)*